when building WebApps and often incorporates many of the other pattern categories just mentioned. *Mobile patterns* describe commonly encountered problems when developing solutions for mobile platforms. At a lower level of abstraction, *idioms* describe how to implement all or part of a specific algorithm or data structure for a software component within the context of a specific programming language. Don't force a pattern, even if it addresses the problem at hand. If the context and forces are wrong, look for another pattern.

In their seminal book on design patterns, Gamma and his colleagues[2] [Gam95] focus on three types of patterns that are particularly relevant to object-oriented design: creational patterns, structural patterns, and behavioral patterns.

INFO



Creational, Structural, and Behavioral Patterns

A wide variety of design patterns that fit into creational, structural, and behavioral categories have been proposed and can be found on the Web. Here is a sampling of patterns for each type. Comprehensive descriptions of each of these patterns can be obtained via links at **www.wikipedia.org.**

Creational Patterns

- **Abstract factory pattern.** Centralize decision of what factory to instantiate.
- **Factory method pattern.** Centralize creation of an object of a specific type, choosing one of several implementations.
- **Builder pattern.** Separate the construction of a complex object from its representation so that the same construction process can create different representations.

Structural Patterns

- **Adapter pattern.** "Adapts" one interface for a class into one that a client expects.
- **Aggregate pattern.** A version of the composite pattern with methods for aggregation of children.
- **Composite pattern.** A tree structure of objects where every object has the same interface.
- **Container pattern.** Create objects for the sole purpose of holding other objects and managing them.
- **Proxy pattern.** A class functioning as an interface to another thing.
- **Pipes and filters.** A chain of processes where the output of each process is the input of the next.

Behavioral Patterns

- **Chain of responsibility pattern.** Command objects are handled or passed on to other objects by logic-containing processing objects.
- **Command pattern.** Command objects encapsulate an action and its parameters.
- **Iterator pattern.** Iterators are used to access the elements of an aggregate object sequentially without exposing its underlying representation.
- **Mediator pattern.** Provides a unified interface to a set of interfaces in a subsystem.
- **Visitor pattern.** A way to separate an algorithm from an object.
- **Hierarchical visitor pattern.** Provides a way to visit every node in a hierarchical data structure such as a tree.

2 Gamma and his colleagues [Gam95] are often referred to as the "Gang of Four" (GoF) in patterns literature.

Creational patterns focus on the "creation, composition, and representation" of objects and provide mechanisms that make the instantiation of objects easier within a system and enforce "constraints on the type and number of objects that can be created within a system" [Maa07]. *Structural patterns* focus on problems and solutions associated with how classes and objects are organized and integrated to build a larger structure. *Behavioral patterns* address problems associated with the assignment of responsibility between objects and the way communication is affected between objects.

14.1.2 Frameworks

Patterns themselves may not be sufficient to develop a complete design. In some cases, it may be necessary to provide an implementation-specific skeletal infrastructure, called a *framework,* for design work. A *framework* is a reusable "mini-architecture" that serves as a foundation from which other design patterns can be applied. That is, you can select a "*reusable mini-architecture* that provides the generic structure and behavior for a family of software abstractions, along with a context . . . which specifies their collaboration and use within a given domain" [Amb98].

A framework is not an architectural pattern, but rather a skeleton with a collection of "plug points" (also called *hooks* and *slots*) that enable it to be adapted to a specific problem domain. The plug points enable you to integrate problem-specific classes or functionality within the skeleton. In an object-oriented context, a framework is a collection of cooperating classes.

Gamma and his colleagues [Gam95] note that patterns are more abstract than frameworks. A framework can be "embodied in code," while a pattern is generally code independent. A framework often encompasses more than a single pattern and is therefore a larger architectural element than a pattern. Finally, a framework resides within a specific application domain, but patterns can be applied in any domain in which the problem to be addressed is encountered.

The designer of a framework will argue that one, reusable, mini-architecture is applicable to all software to be developed within a limited domain of application. To be most effective, frameworks are applied with no changes. Additional design elements may be added, but only via the plug points that allow the designer to flesh out the framework skeleton.

14.1.3 Describing a Pattern

Pattern-based design begins with the recognition of patterns within the application you intend to build, continues with a search to determine whether others have addressed the pattern, and concludes with the application of an appropriate pattern to the problem at hand. The second of these three tasks is often the most difficult. How do you find patterns that fit your needs?

An answer to this question must rely on effective communication of the problem the pattern addresses, the context in which the pattern resides, the system of forces that mold the context, and the solution that is proposed. To communicate this information unambiguously, a standard form or template for pattern descriptions is required. Although several different pattern templates have been proposed, almost all contain a major

subset of the content suggested by Gamma and his colleagues [Gam95]. A simplified pattern template is shown in the sidebar.

INFO



Design Pattern Template

Pattern name. Describes the essence of the pattern in a short but expressive name.

Problem. Describes the problem that the pattern addresses.

Motivation. Provides an example of the problem.

Context. Describes the environment in which the problem resides, including the application domain.

Forces. Lists the system of forces that affect the way the problem must be solved; includes a discussion of limitation and constraints that must be considered.

Solution. Provides a detailed description of the solution proposed for the problem.

Intent. Describes the pattern and what it does.

Collaborations. Describes how other patterns contribute to the solution.

Consequences. Describes the potential trade-offs that must be considered when the pattern is implemented and the consequences of using the pattern.

Implementation. Identifies special issues that should be considered when implementing the pattern.

Known uses. Provides examples of actual uses of the design pattern in real applications.

Related patterns. Cross-references related design patterns.

The names of design patterns should be chosen with care. One of the key technical problems in pattern-based design is the inability to find existing patterns when hundreds or thousands of candidate patterns exist. The search for the "right" pattern is aided immeasurably by a meaningful pattern name.

A pattern template provides a standardized means for describing a design pattern. Each of the template entries represents characteristics of the design pattern that can be searched (e.g., via a database) so that the appropriate pattern can be found.

14.1.4 Machine Learning and Pattern Discovery

Software patterns can be described as best practice solutions to known problems. Information about where design patterns have been implemented in a software design is useful information for developers to have when creating or maintaining a software system. Sadly, this information is lost due to poor documentation practices of the original developers. In recent years, there has been significant interest in making use of automatic techniques to identify new patterns present, but not documented, in existing software products [Alh12].

One way to do this is to create an artificial intelligence (AI) system capable of recognizing design patterns after examining many similar software systems. The same software pattern may be implemented in many ways. *Machine learning*[3] techniques

3 Machine learning is an AI technique that uses statistical techniques to allow a system to learn from examples and improve its performance without being explicitly programmed. For those readers interested in pursuing this subject in greater detail, see [Kub17].

can provide a means of teaching a system to recognize the presence of a pattern in the software source code. The machine learning system repeatedly examines a training set containing both good and bad examples of software patterns using specific quantitative criteria of "good" and "bad" examples. This process continues until the system has learned to recognize most of the good patterns in the training set. Often these training sets are derived from large open-source software systems available on the Internet [Zan15].

Once trained, the tool can be used to locate software patterns in new systems outside the training set. To be useful, the software patterns are collected in a repository. Ideally, this repository can be searched for software patterns applicable to the problems developers need to solve [Amp13].

14.2 PATTERN-BASED SOFTWARE DESIGN

The best designers in any field have an uncanny ability to see patterns that characterize a problem and the corresponding patterns that can be combined to create a solution. Throughout the design process, you should look for every opportunity to apply existing design patterns (when they meet the needs of the design) rather than creating new ones.

14.2.1 Pattern-Based Design in Context

Pattern-based design is not used in a vacuum. The concepts and techniques discussed for architectural, component-level, and user interface design (Chapter 10 through 12) are all used in conjunction with a pattern-based approach.

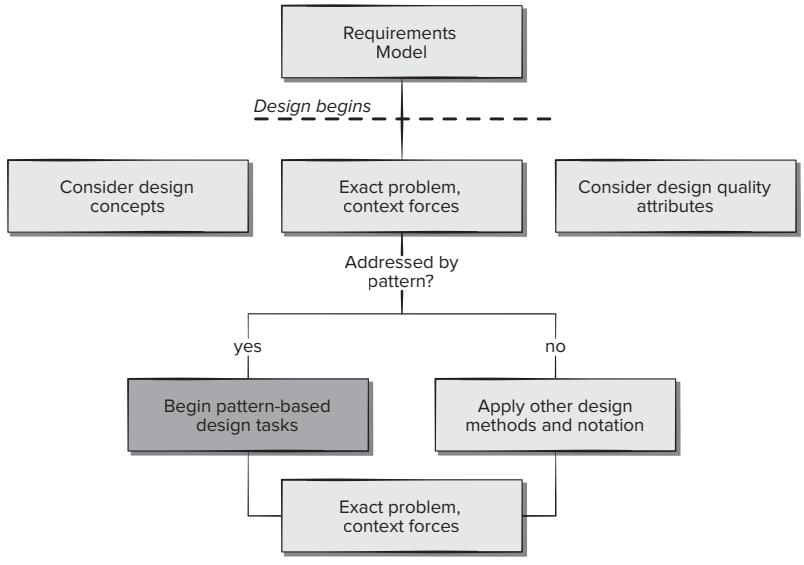
In Chapter 9, we noted that a set of quality guidelines and attributes serve as the basis for all software design decisions. The decisions themselves are influenced by a set of fundamental design concepts (e.g., separation of concerns, stepwise refinement, functional independence) that are achieved using heuristics that have evolved over many decades, and best practices (e.g., techniques, modeling notation) that have been proposed to make design easier to perform and more effective as a basis for construction.

The role of pattern-based design in all of this is illustrated in Figure 14.1. A software designer begins with a requirements model (either explicit or implied) that presents an abstract representation of the system. The requirements model describes the problem set, establishes the context, and identifies the system of forces that hold sway. It may imply the design in an abstract manner, but the requirements model does little to represent the design explicitly.

As you begin your work as a designer, it's always important to keep quality attributes (Chapter 9) in mind. These attributes establish a way to assess software quality but do little to help you achieve it. Therefore, you should apply proven techniques for translating the abstractions contained in the requirements model into a more concrete form that is the software design. To accomplish this, you'll use the methods and modeling tools available for architectural, component-level, and interface design. But, only when you're faced with a problem, context, and system of forces that have not been solved before. If a solution already exists, use it! And that means applying a pattern-based design approach.

FIGURE 14.1

Pattern-based design in context



14.2.2 Thinking in Patterns

Pattern-based design implies a "new way of thinking" [Sha05] that begins by considering context—the big picture. As context is evaluated, you extract a hierarchy of problems that must be solved. Some of these problems will be global in nature, while others will address specific features and functions of the software. All will be affected by a system of forces that will influence the nature of the solution that is proposed.

Shalloway and Trott [Sha05] suggest the following approach[4] that enables a designer to think in patterns:

1. Be sure you understand the big picture—the context in which the software to be built resides. The requirements model should communicate this to you.
2. Examining the big picture, extract the patterns that are present at that level of abstraction.
3. Begin your design with "big picture" patterns that establish a context or skeleton for further design work.
4. "Work inward from the context" [Sha05], looking for patterns at lower levels of abstraction that contribute to the design solution.
5. Repeat steps 1 to 4 until the complete design is fleshed out.
6. Refine the design by adapting each pattern to the specifics of the software you're trying to build.

It's important to note that patterns are not independent entities. Design patterns that are present at a high level of abstraction will invariably influence the ways other patterns are applied at lower levels of abstraction. In addition, patterns often collaborate

4 Based on the work of Christopher Alexander [Ale79].

with one another. The implication is that when you select an architectural pattern, it may very well influence the component-level design patterns you choose. Likewise, when you select a specific interface design pattern, you are sometimes forced to use other patterns that collaborate with it.

To illustrate, consider the **SafeHomeAssured.com** WebApp. If you consider the big picture, the WebApp must address how to provide information about *SafeHome* products and services, how to sell *SafeHome* products and services to customers, and how to establish Internet-based monitoring and control of an installed security system. Each of these fundamental problems can be further refined into a set of subproblems.

For example, *How to sell* via the Internet implies an **e-commerce** pattern that itself implies many patterns at lower levels of abstraction. The **e-commerce** pattern (likely, an architectural pattern) implies mechanisms for setting up a customer account, displaying the products to be sold, selecting products for purchase, and so forth. Hence, if you think in patterns, it is important to determine whether a pattern for setting up an account exists. If **SetUpAccount** is available as a viable pattern for the problem context, it may collaborate with other patterns such as **BuildInputForm, ManageFormsInput,** and **ValidateFormsEntry.** Each of these patterns delineates problems to be solved and solutions that may be applied.

14.2.3 Design Tasks

The following design tasks are applied when a pattern-based design philosophy is used:

1. **Examine the requirements model and develop a problem hierarchy.** Describe each problem and subproblem by isolating the problem, the context, and the system of forces that apply. Work from broad problems (high level of abstraction) to smaller subproblems (at lower levels of abstraction).
2. **Determine if a reliable pattern language has been developed for the problem domain.** A *pattern language* encompasses a collection of patterns, each described using a standardized template (Section 14.1.3) and interrelated to show how these patterns collaborate to solve problems across an application domain. The *SafeHome* software team would look for a pattern language developed specifically for home security products. If that level of pattern language specificity could not be found, the team would partition the *SafeHome* software problem into a series of generic problem domains (e.g., digital device monitoring problems, user interface problems, digital video management problems) and search for appropriate pattern languages.
3. **Beginning with a broad problem, determine whether one or more architectural patterns are available for it.** If an architectural pattern is available, be certain to examine all collaborating patterns. If the pattern is appropriate, adapt the design solution proposed and build a design model element that adequately represents it. For example, a broad problem for the **SafeHomeAssured.com** WebApp is addressed with an **e-commerce** pattern (Section 14.2.2). This pattern will suggest a specific architecture for addressing e-commerce requirements.
4. **Using the collaborations provided for the architectural pattern, examine subsystem or component-level problems and search for appropriate patterns to address them.** It may be necessary to search through other pattern repositories as